



UNIVERSITÀ DI PISA

Artificial Intelligence and Data Engineering

Machine Learning and Data Mining

**Project Documentation of**  
***FakeNews Detector***

Authors:

Alessio Grillo - Andrea Giacomazzi

Academic Year: 2025-2026

# Contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>1</b>  |
| <b>2</b> | <b>Data Analysis</b>                                                | <b>2</b>  |
| 2.1      | Dataset Overview and Structure . . . . .                            | 2         |
| 2.2      | Class Distribution . . . . .                                        | 2         |
| 2.3      | Exploratory Data Analysis (EDA) . . . . .                           | 3         |
| 2.3.1    | Data Integrity and Missing Values . . . . .                         | 3         |
| 2.3.2    | Text Length Statistics . . . . .                                    | 4         |
| 2.3.3    | Text Length Distribution Analysis . . . . .                         | 4         |
| 2.3.4    | Unigram Frequency Analysis . . . . .                                | 5         |
| 2.3.5    | Bigram Frequency Analysis . . . . .                                 | 6         |
| 2.3.6    | Stylometric Distribution Anomalies . . . . .                        | 7         |
| <b>3</b> | <b>Data Pre-Processing</b>                                          | <b>8</b>  |
| 3.1      | Data Cleaning and Integration . . . . .                             | 8         |
| 3.2      | Impact of Row Deletion . . . . .                                    | 8         |
| 3.3      | Justification for Data Retention: Handling Missing Titles . . . . . | 9         |
| 3.4      | Stylometric Feature Engineering . . . . .                           | 9         |
| 3.5      | Text Normalization, Tokenization, and Leakage Prevention . . . . .  | 10        |
| 3.6      | Vectorization: 10,000 TF-IDF Features . . . . .                     | 10        |
| 3.7      | Stylometric Feature Scaling . . . . .                               | 11        |
| <b>4</b> | <b>Methodology for Model Explainability: SHAP and LIME</b>          | <b>12</b> |
| 4.1      | Global Interpretability with SHAP . . . . .                         | 12        |
| 4.2      | Local Interpretability with LIME . . . . .                          | 13        |
| <b>5</b> | <b>Models evaluation and comparison</b>                             | <b>14</b> |
| 5.1      | Candidate Algorithms . . . . .                                      | 14        |
| 5.2      | Performance Metrics . . . . .                                       | 14        |
| 5.3      | Confusion Matrices . . . . .                                        | 15        |
| 5.4      | Metric values . . . . .                                             | 15        |
| <b>6</b> | <b>Model Selection and Hyperparameter Tuning</b>                    | <b>17</b> |
| 6.1      | Hyperparameter Optimization . . . . .                               | 20        |

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>7</b> | <b>Conclusion and Future Works</b>        | <b>23</b> |
| 7.1      | Conclusion . . . . .                      | 23        |
| 7.2      | Future Works . . . . .                    | 24        |
| <b>A</b> | <b>User Manual</b>                        | <b>25</b> |
| A.1      | Operational Logic . . . . .               | 25        |
| A.2      | Online Detection (URL Mode) . . . . .     | 25        |
| A.3      | Offline Detection (Manual Mode) . . . . . | 25        |
| A.4      | Installation . . . . .                    | 25        |
| A.5      | Running the Application . . . . .         | 26        |
| A.6      | Application Mock-ups . . . . .            | 27        |

# Chapter 1

## Introduction

In the contemporary digital era, the democratization of information has led to an unprecedented proliferation of data. However, this growth has been accompanied by the rapid spread of **misinformation** and "fake news," which pose significant threats to social cohesion, political stability, and public trust.

The detection of deceptive content is no longer a task that can be managed solely by human fact-checkers due to the sheer volume and velocity of online discourse. This project introduces *FakeNews Detector*, an automated detection system designed to identify and flag non-credible information. By leveraging **Natural Language Processing (NLP)** and **Machine Learning** algorithms, our software analyzes linguistic patterns and source metadata to provide a reliability score, offering a robust tool in the fight against digital deception.

To achieve this, we performed a comparative assessment of multiple models using key metrics to determine the optimal approach to detect fake news.

## Chapter 2

# Data Analysis

In order to develop a robust fake news detector, we utilize the **WELFake dataset**, a comprehensive collection of news articles sourced from Kaggle. This dataset is a merged compilation of several smaller datasets (including BuzzFeed, LIAR, and others), making it one of the largest public resources for misinformation research.

### 2.1 Dataset Overview and Structure

The dataset consists of 72,134 records, distributed across four primary features. The structure of the data is summarized in Table 2.1.

| Feature       | Type    | Description                            |
|---------------|---------|----------------------------------------|
| <i>Serial</i> | Integer | Unique identifier for each entry.      |
| <i>Title</i>  | String  | The headline of the news article.      |
| <i>Text</i>   | String  | The body content of the article.       |
| <i>Label</i>  | Binary  | Target variable: 1 (Real) or 0 (Fake). |

Table 2.1: WELFake Dataset Features

### 2.2 Class Distribution

A critical aspect of data analysis is checking for **class balance**. An imbalanced dataset can lead to biased models that favor the majority class. The WELFake dataset is remarkably well-balanced:

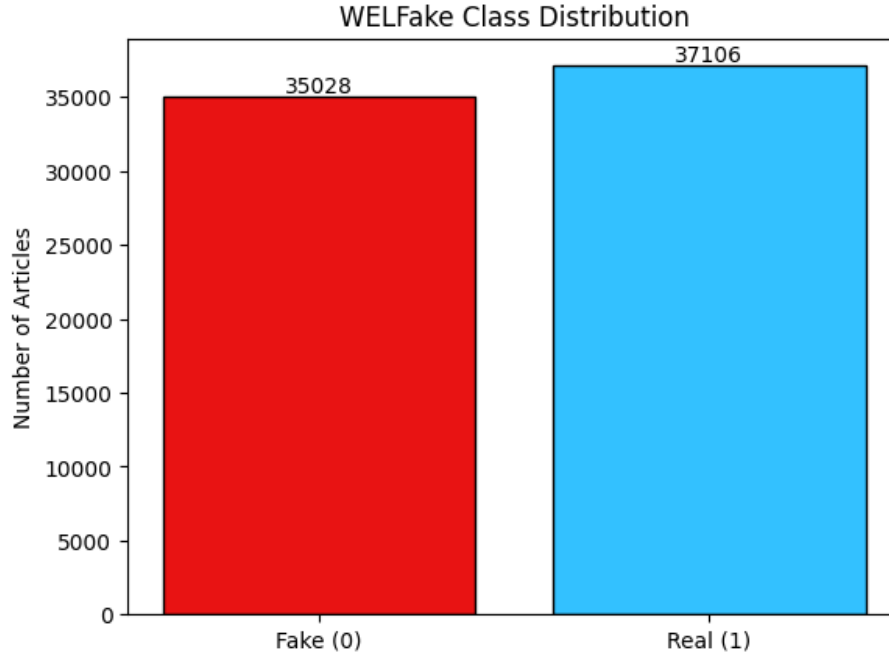


Figure 2.1: WELFake Class Distribution

The bar chart illustrates the distribution of labels within the WELFake dataset. As shown, the dataset is nearly perfectly balanced, with *Fake News* ( $n=37,108$ ) representing approximately 51.5% of the data and *Real News* ( $n=35,026$ ) making up the remaining 48.5%. This near-equal distribution ensures that the classification metrics, such as accuracy and F1-score, will be reliable indicators of model performance without the need for oversampling or undersampling techniques.

## 2.3 Exploratory Data Analysis (EDA)

Initial exploration reveals several key linguistic characteristics:

### 2.3.1 Data Integrity and Missing Values

Preliminary analysis identified a small number of missing values (NaN) within the *Title* and *Text* columns. Specifically, there are 558 missing titles and 39 missing text.

### 2.3.2 Text Length Statistics

Statistical analysis of article lengths shows a significant variance between samples. On average:

- **Titles** range from 5 to 30 words.
- **Article Bodies** average approximately 500 to 800 words, though some entries exceed several thousand words.

Interestingly, *fake* news articles often exhibit specific patterns, such as overly sensationalist titles (frequent use of capital letters and exclamation marks) or shorter, more repetitive body text compared to investigative journalism found in the *real* news category.

### 2.3.3 Text Length Distribution Analysis

To further evaluate the structural properties of the corpus, a Kernel Density Estimate (KDE) was computed for the article lengths, measured by word count (Figure 2.2).

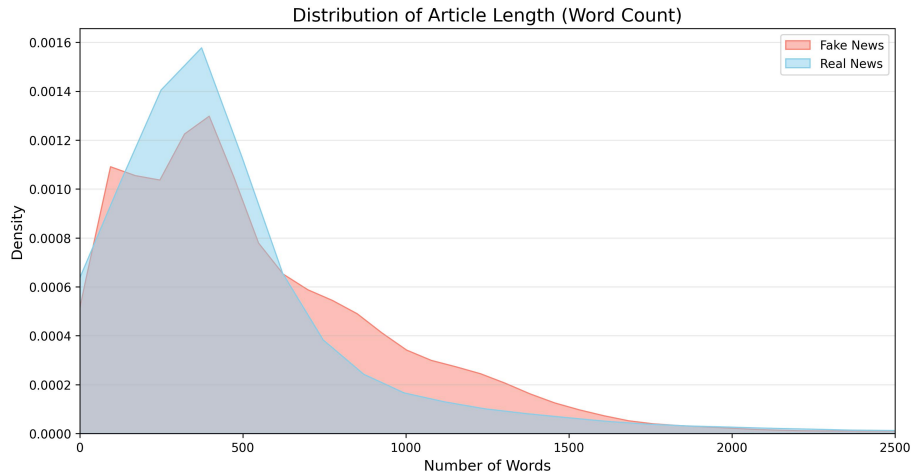


Figure 2.2: KDE Plot of Word Count Distribution: comparison between Fake and Real News.

The distribution reveals a significant divergence in writing patterns. *Real News* articles exhibit a sharp, localized peak around 400 words, reflecting the standardized editorial formats typical of professional journalism. Conversely, the *Fake News* distribution shows a "heavier tail," with a higher density of articles ranging from 750 to 1,500 words.

This suggests that while authentic news follows a concise and structured reporting style, deceptive content in this dataset often relies on longer, more

verbose narratives. From a machine learning perspective, this confirms that **text length** is a discriminative feature: the classifier can leverage these statistical differences in document structure to aid the distinction between the two classes.

### 2.3.4 Unigram Frequency Analysis

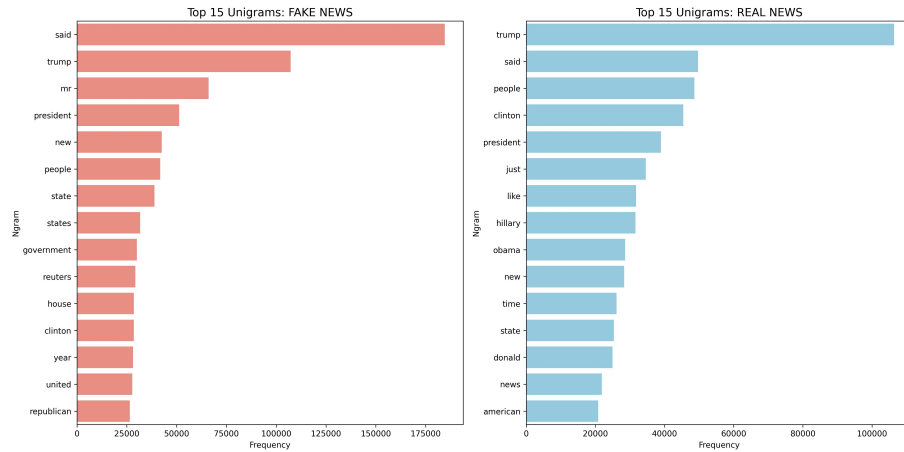


Figure 2.3: Frequency distribution of the top 15 unigrams for Fake News (left) and Real News (right).

Figure 2.3 illustrates the frequency distribution of the top 15 unigrams across the *Fake* and *Real* news categories. A comparative analysis of these charts reveals crucial insights into the dataset’s linguistic landscape and highlights potential risks of data leakage.

First, there is a prominent thematic overlap between the two classes. Political entities and terms such as “*trump*”, “*president*”, and “*clinton*” dominate both distributions. This underscores the dataset’s heavy concentration on US politics, indicating that the mere presence of these entities is not a reliable discriminator for veracity.

However, the most striking anomaly is the disproportionate frequency of the token “*said*” in the *Fake News* corpus, which exceeds 175,000 occurrences. This volume is more than triple its frequency in the authentic news category. Such an extreme prevalence suggests a highly repetitive, mechanical narrative structure, likely stemming from an over-reliance on fabricated quotations or a less sophisticated journalistic style. Furthermore, the high rank of the courtesy title “*mr*” in the deceptive class further confirms this stylistic rigidity.

Crucially, the analysis exposes direct evidence of source-mimicking. The token “*reuters*” appears as the 10th most frequent unigram within the *Fake News* subset. This indicates a deliberate tactic by malicious authors to artificially inflate the credibility of their articles by falsely citing a reputable news agency.



From a machine learning perspective, these findings are highly actionable. If left unaddressed, the classifier would likely overfit to these procedural tokens (e.g., “*said*”) and publisher signatures (e.g., “*reuters*”), rather than learning the underlying semantics of deception. Consequently, this exploratory analysis provides the empirical justification for the implementation of the custom stopword filtering discussed in the pre-processing phase.

### 2.3.5 Bigram Frequency Analysis

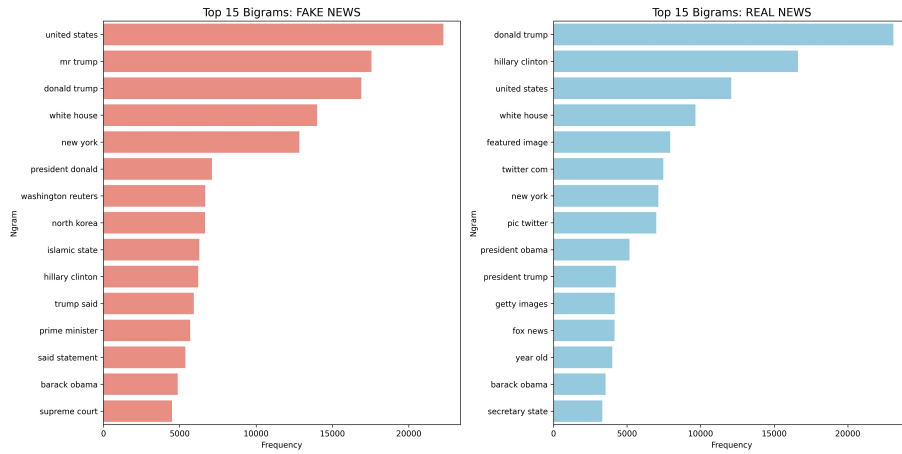


Figure 2.4: Frequency distribution of the top 15 bigrams for Fake News (left) and Real News (right).

While unigrams help identify high-frequency noise, analyzing the distribution of bigrams (Figure 2.4) provides deeper insights into the narrative context and structural formatting of the articles. This level of analysis is crucial for detecting **contextual leakage** and platform-specific artifacts that could severely skew the classifier’s performance.

A primary observation from the *Real News* distribution is the prominent presence of metadata and social media residuals. Bigrams such as “*featured image*”, “*twitter com*”, “*pic twitter*”, and “*getty images*” rank highly among authentic articles. These sequences are not indicative of journalistic truth; rather, they are technical remnants of the web scraping process and standard editorial formatting for multimedia content. If retained, the model would likely learn to associate these specific web tags with the positive class, resulting in an artificial inflation of accuracy based on HTML formatting rather than semantic veracity.

Conversely, the *Fake News* distribution reveals direct evidence of **source-mimicking**. The appearance of the bigram “*washington reuters*” highlights a common deceptive tactic where malicious authors prepend the datelines of legitimate news hubs to fabricate credibility. Allowing the model to process this

bigram would inadvertently train it to recognize a copied signature rather than the underlying linguistic patterns of disinformation.

Furthermore, both categories share a heavy concentration of political entities (e.g., “donald trump”, “hillary clinton”, “white house”, “united states”). This confirms a strong domain bias toward US politics within the corpus.

To ensure the classifier learns generalizable stylistic features rather than memorizing metadata, web structures, or specific political entities, these identified bigrams—both the technical residuals and the mimicked signatures—were systematically added to the custom stopwords exclusion list prior to the TF-IDF vectorization phase.

### 2.3.6 Stylometric Distribution Anomalies

The boxplot analysis of stylistic features (Figure 2.5) reveals a counter-intuitive distribution compared to general assumptions in fake news detection literature.

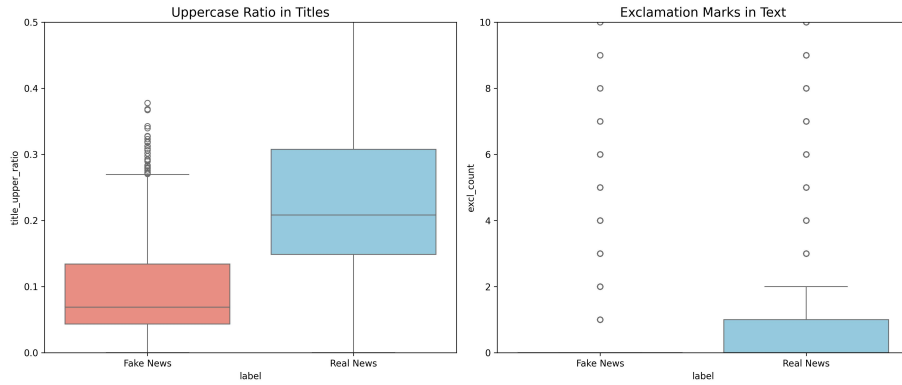


Figure 2.5: Boxplot of Uppercase Ratio and Exclamation Mark frequency. Contrary to expectations, authentic articles exhibit a higher density of these markers within this specific dataset.

The *Uppercase Ratio* in titles is higher for *Real News*, which can be attributed to the professional use of Title Case and institutional acronyms common in agency reporting. Conversely, while *Fake News* shows a lower median, it presents a significant number of extreme outliers, representing a subset of highly sensationalist headlines.

Regarding *Exclamation Marks*, the data shows that deceptive content in the WELFake corpus adopts a surprisingly sober tone, likely to mimic the perceived authority of formal reports. Authentic news, instead, shows a higher frequency of punctuation marks due to the presence of verbatim quotes from speeches and interviews. These findings underscore the complexity of the dataset and justify the use of a non-linear classifier like **LightGBM**, which can capture these subtle, non-obvious distinctions that traditional linear models might miss.

## Chapter 3

# Data Pre-Processing

The transformation of the raw WELFake dataset into a structured format involves a multi-stage pipeline. This process ensures that the noise inherent in web-scraped news is minimized while the discriminative features are amplified.

### 3.1 Data Cleaning and Integration

Before any linguistic analysis, the dataset underwent a rigorous cleaning phase to ensure structural integrity:

- **Handling Missing Values:** Entries with null values in the *title* or *text* fields were identified. Given the size of the dataset, these rows were removed to maintain a high-quality corpus.
- **Feature Integration:** We performed a **horizontal integration** of the *title* and *text* columns. This creates a unified "content" feature, allowing the model to analyze headlines (often rich in clickbait patterns) alongside the full body of the article.

### 3.2 Impact of Row Deletion

To ensure the highest data quality, we performed a *listwise deletion* of rows containing null values in the text field. This resulted in the removal of 39 records, accounting for 0.05% of the total corpus (72,134 samples), finally obtaining a dataset of 72,095 total samples. We justified this decision based on two primary factors:

1. **Preservation of Class Balance:** The original dataset followed a near 51.5/48.5 distribution between *Fake* and *Real* news. The deletion of these sparse records did not statistically alter this ratio. The final distribution remains balanced (approx. 51.5% Fake and 48.5% Real), ensuring that the classification metrics remain unbiased.

2. **Avoidance of Synthetic Noise:** Filling missing titles or texts with placeholders (e.g., "Unknown") could introduce artificial patterns that do not exist in real-world journalism, potentially leading the model to associate "missing information" with a specific class.

### 3.3 Justification for Data Retention: Handling Missing Titles

Despite the presence of 558 missing entries in the *Title* column, we deliberately chose not to discard these records, opting instead for a concatenation strategy. This decision is supported by the following methodological considerations:

- **Information Preservation:** In the context of NLP-based fake news detection, the *Text* body contains the vast majority of semantic and stylistic markers. An article without a title still provides substantial data for the TF-IDF vectorizer to extract meaningful features. Discarding these rows would result in an unnecessary loss of 558 valid linguistic samples.
- **Statistical Power:** Retaining these entries maximizes the training pool for our 10-fold cross-validation, ensuring that the model is exposed to a broader variety of news structures, including those that may lack conventional headlines in their original scraped format.

### 3.4 Stylometric Feature Engineering

While the text was normalized for semantic analysis, we simultaneously extracted raw stylistic markers to capture the "anatomy" of misinformation. This extraction was performed *prior* to text normalization, as lowercasing and noise removal would destroy these syntactic nuances:

**Lexical Casing:** We calculated the percentage of **uppercase words**. High frequencies of capitalized text are a common proxy for sensationalism.

**Article Volumetry:** We integrated the **size of the article** (total word count), as fake news often lacks the depth and length of authentic journalism.

**Linguistic Integrity:** A dedicated module quantified the number of **spelling errors**, distinguishing professionally edited news from unverified content.

**Punctuation Analysis:** We tracked the density of **exclamation (!)** and **question marks (?)** to detect inflammatory or non-neutral rhetoric.

### 3.5 Text Normalization, Tokenization, and Leakage Prevention

To reduce the complexity of the natural language and prevent data leakage, we applied a series of normalization steps using the NLTK library:

1. **Noise Removal:** We stripped non-alphanumeric characters using regular expressions.
2. **Case Folding:** All text was converted to lowercase to unify the vocabulary.
3. **Tokenization:** The clean text was segmented into individual **tokens** (words), which serve as the fundamental units for the subsequent vectorization.
4. **Lemmatization:** To group inflected forms of words, tokens were reduced to their morphological roots (e.g., "running" to "run"), thereby reducing the dimensionality of the feature space without losing semantic meaning.
5. **Custom Stopword Elimination:** Beyond standard English stopwords (e.g., "of", "the", "in"), we implemented a targeted exclusion list based on our exploratory N-gram analysis. Terms such as "reuters", "twitter com", and "featured image" were actively removed. Retaining these platform-specific artifacts and source-mimicking signatures would introduce severe **contextual leakage**, leading the classifier to recognize metadata formatting rather than the underlying linguistic patterns of disinformation.

### 3.6 Vectorization: 10,000 TF-IDF Features

For the numerical representation of the text, we implemented the **TF-IDF (Term Frequency-Inverse Document Frequency)** method. We configured the vectorizer to extract the **10,000 most relevant features** (unigrams and bigrams). This threshold is a deliberate strategy to counteract the *curse of dimensionality*. In high-dimensional spaces, the volume of the space increases so exponentially that the available data (72,095 samples) becomes statistically sparse. This sparsity makes it difficult for classifiers to identify meaningful clusters, as all observations appear equidistant and isolated.

By filtering out the long tail of infrequent n-grams and focusing on the most informative 10,000 dimensions, we ensure a higher **data-to-feature ratio**. This condensation forces the model to prioritize high-signal linguistic patterns, thereby reducing the risk of *overfitting*.

Crucially, to maintain strict experimental integrity and prevent data leakage, the TF-IDF vectorizer was fitted **exclusively on the training partition** of each fold during the cross-validation process. The derived vocabulary was then used to transform the validation fold, ensuring the model had zero prior knowledge of the test data distribution.

### 3.7 Stylometric Feature Scaling

To ensure that the stylometric features (such as text length, uppercase ratio, and punctuation frequency) contribute equally to the model’s decision-making process alongside the TF-IDF vector, a normalization step is required. Since raw stylometric counts and ratios vary across different ranges, we apply the `MinMaxScaler` to transform these features.

The `MinMaxScaler` rescales each feature into a fixed range of  $[0, 1]$ . This is particularly important because the TF-IDF features are inherently normalized within a similar magnitude. By aligning the scales, we prevent features with larger absolute values (e.g., character count) from dominating those with smaller values (e.g., exclamation mark ratio).

Following the same rigorous zero-knowledge methodology applied to the TF-IDF vectorization, the scaling of stylometric features was performed **strictly within the cross-validation loop**. The scaler was fitted on the training set of each fold and subsequently applied to the validation set.

The transformation is defined by the following equation for each feature  $x$ :

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (3.1)$$

where  $x'$  is the normalized value, and  $\min(x)$  and  $\max(x)$  are the minimum and maximum values of the feature found exclusively in the training dataset. After this scaling process, the resulting five-dimensional stylistic vector is horizontally stacked with the sparse TF-IDF matrix to form the final feature set for the classifier.

## Chapter 4

# Methodology for Model Explainability: SHAP and LIME

To ensure the transparency and reliability of the **FakeNews Detector**, a hybrid Explainable AI (XAI) approach was implemented. This strategy distinguishes between *global interpretability* for model validation and *local interpretability* for end-user interaction, utilizing SHAP and LIME respectively.

### 4.1 Global Interpretability with SHAP

During the model evaluation and validation phase, SHAP (SHapley Additive exPlanations) was applied to the entire test set. Grounded in cooperative game theory, SHAP provides a mathematically rigorous method to assign an importance value to each feature (word) for every prediction.

**Motivations for using SHAP on the test set:**

- **Axiomatic Foundation:** SHAP guarantees properties such as local accuracy and consistency, providing a reliable gold standard for understanding which terms (e.g., specific sensationalist adjectives) the model relies on across thousands of examples.
- **Feature Interaction:** It accounts for the interaction between words, allowing us to see how the presence of one term modifies the impact of another.
- **Model Debugging:** By analyzing the global summary plots, it was possible to confirm that the model learned meaningful linguistic patterns rather than noise or dataset biases.

## 4.2 Local Interpretability with LIME

For the real-time classification of single news articles within the application, LIME (Local Interpretable Model-agnostic Explanations) was selected as the primary explanation engine.

### Motivations for using LIME in the application:

- **Computational Efficiency:** Unlike SHAP, which can be computationally expensive and slow for high-dimensional text data, LIME is significantly faster. It generates a local surrogate model around the specific input in seconds, ensuring a responsive user experience.
- **Intuitive Visualization:** LIME is exceptionally effective at highlighting specific words in the original text, making it easy for the user to see exactly which parts of the news article triggered a “Fake” or “Real” verdict.
- **User-Centric Focus:** While SHAP explains the model’s overall logic, LIME focuses exclusively on the *why* of the current prediction, which is the primary concern of the end-user.



## Chapter 5

# Models evaluation and comparison

To identify the optimal classifier, we used six distinct algorithms, evaluated through a **Stratified 10-fold Cross-Validation** to ensure statistical robustness and mitigate overfitting.

### 5.1 Candidate Algorithms

- **Logistic Regression (LR):** A baseline linear model efficient for high-dimensional sparse data.
- **Multinomial Naive Bayes (MNB):** A probabilistic framework ideal for text classification.
- **Linear Support Vector Machines (SVM):** Effective at finding the optimal separating hyperplane in the 10,000-dimensional space.
- **Random Forest (RF):** A bagging ensemble that captures non-linear relationships and is robust against outliers.
- **AdaBoost:** A boosting technique that iteratively corrects the errors of weak learners.
- **LightGBM:** A high-performance gradient boosting framework optimized for speed and large-scale datasets.

### 5.2 Performance Metrics

The models were compared using five key metrics derived from the confusion matrix:

- **Accuracy:** The overall rate of correct predictions.

- **Precision:** The ability of the model to avoid false positives (labeling real news as fake).
- **Recall:** The ability of the model to identify all actual fake news (minimizing false negatives).
- **F1-Score:** The harmonic mean of precision and recall, serving as the primary balanced benchmark.
- **AUC-ROC:** The Area Under the Receiver Operating Characteristic curve, measuring the model's threshold-independent ability to discriminate between classes.

## 5.3 Confusion Matrices

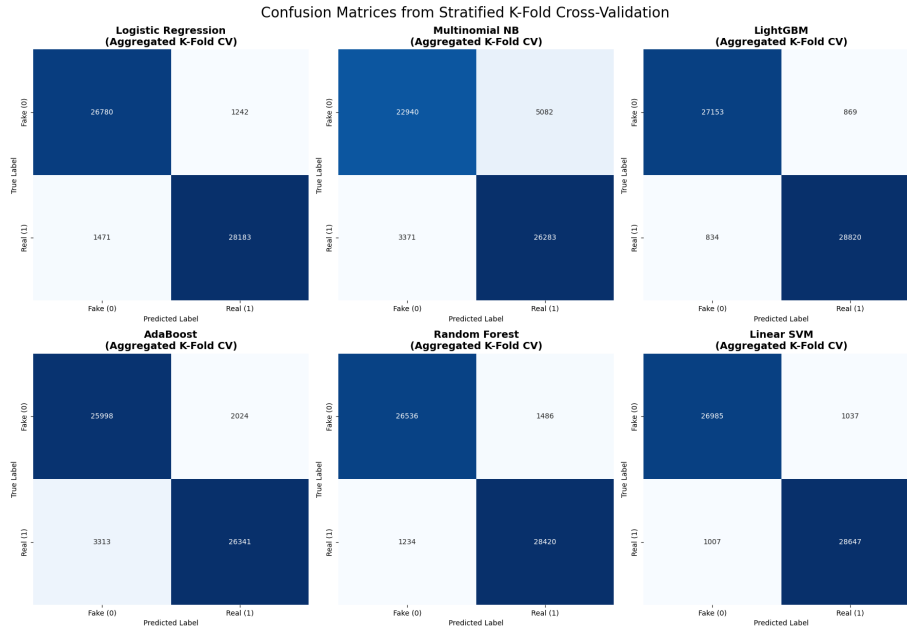


Figure 5.1: Model Confusion Matrices

## 5.4 Metric values

| Metric/Model                   | Accuracy | Precision | Recall | F1-Score | AUC-ROC |
|--------------------------------|----------|-----------|--------|----------|---------|
| <b>LightGBM</b>                | 0.9705   | 0.9707    | 0.9719 | 0.9713   | 0.9957  |
| <b>Linear SVM</b>              | 0.9646   | 0.9651    | 0.9660 | 0.9656   | 0.9934  |
| <b>Random Forest</b>           | 0.9528   | 0.9503    | 0.9584 | 0.9543   | 0.9909  |
| <b>Logistic Regression</b>     | 0.9530   | 0.9578    | 0.9504 | 0.9541   | 0.9908  |
| <b>AdaBoost</b>                | 0.9075   | 0.9287    | 0.8883 | 0.9080   | 0.9705  |
| <b>Multinomial Naive Bayes</b> | 0.8534   | 0.8380    | 0.8863 | 0.8615   | 0.9282  |

Table 5.1: Performance Metrics across different Machine Learning Models

## Chapter 6

# Model Selection and Hyperparameter Tuning

We partitioned the dataset into an 80% training set and a 20% independent test set. To ensure robust model selection, we employed a stratified 10-fold Cross-Validation framework exclusively on the training partition for the following reasons:

- **Bias-Variance Tradeoff:**  $k = 10$  provides an optimal balance between a model's ability to be accurate (low bias) and its ability to remain unaffected by noise or specific fluctuations in the training data (low variance).
- **Robustness:** With 72,095 samples, 10 folds ensure that the model is tested on diverse subsets, making the final accuracy estimate significantly more reliable than a single split.
- **Generalization:** This approach ensures that our detector generalizes well to unseen news, rather than overfitting to specific patterns in the training data.
- **Class Distribution Preservation (Stratification):** Unlike standard K-Fold, the *stratified* variant ensures that each of the ten folds maintains the same percentage of "Real" and "Fake" samples as the original 72,095-entry corpus. This prevents accidental class imbalances during the training phase, providing a more objective evaluation of the model's discriminative power.

Following a rigorous comparative analysis of multiple machine learning algorithms, **LightGBM** was identified as the top-performing model. The model selection process was based on performance metrics (such as F1-Score) evaluated through stratified 3-fold cross-validation on the validation set.

To interpret the underlying logic of the model and ensure the transparency of its decision-making process, we utilized **SHAP (SHapley Additive exPlanations)** values. The following summary plot illustrates the primary features

that influenced the model's predictions, ranked by their average absolute SHAP values.

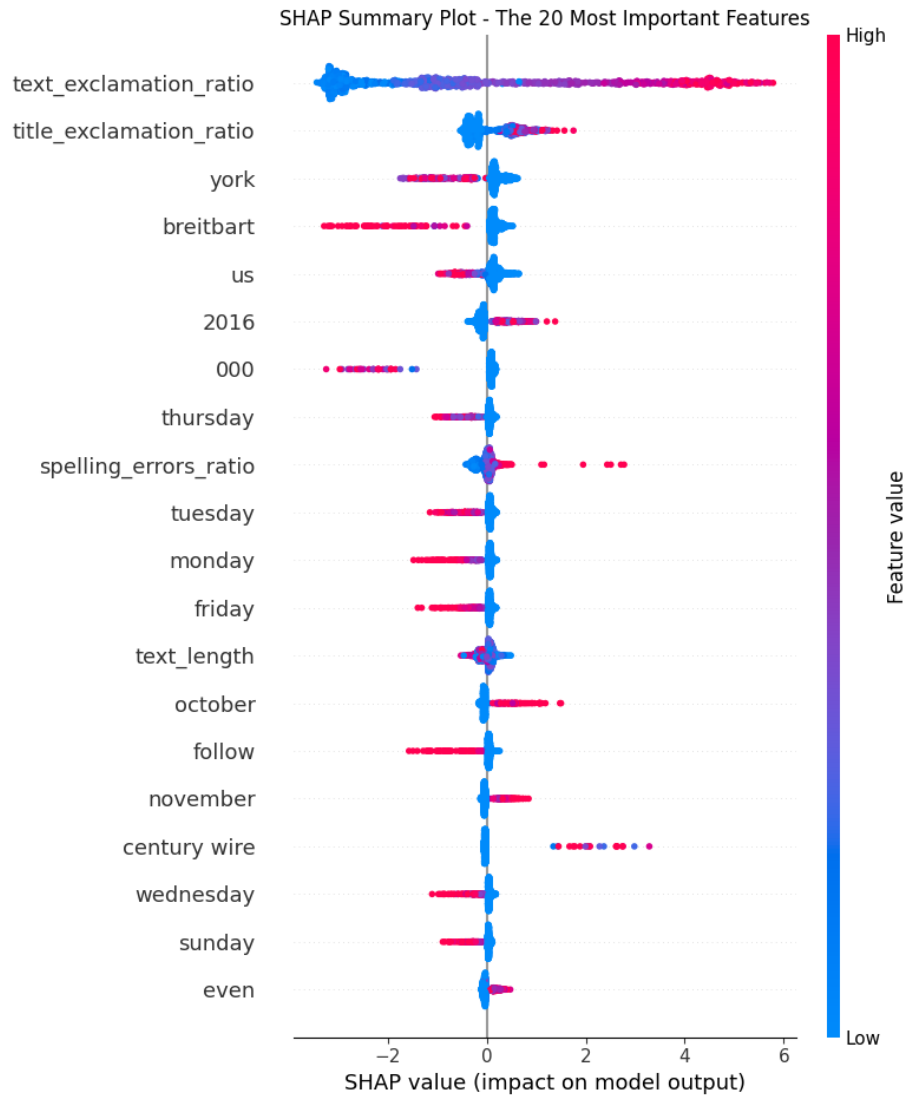


Figure 6.1: SHAP Summary Plot for the LightGBM model, showing feature importance and their impact on model output.

Once the optimal model was selected and its logic verified, we conducted a final benchmark on the **independent test set**. This set was not used during the training or selection phases, providing an unbiased estimate of the model's

generalization capabilities in a real-world scenario. The results are summarized in Table 6.

| Metric    | Value  |
|-----------|--------|
| Accuracy  | 0.9692 |
| Precision | 0.9692 |
| Recall    | 0.9691 |
| F1-Score  | 0.9691 |
| AUC-ROC   | 0.9955 |

Table 6.1: Final Benchmark Results on the Test Set (LightGBM)

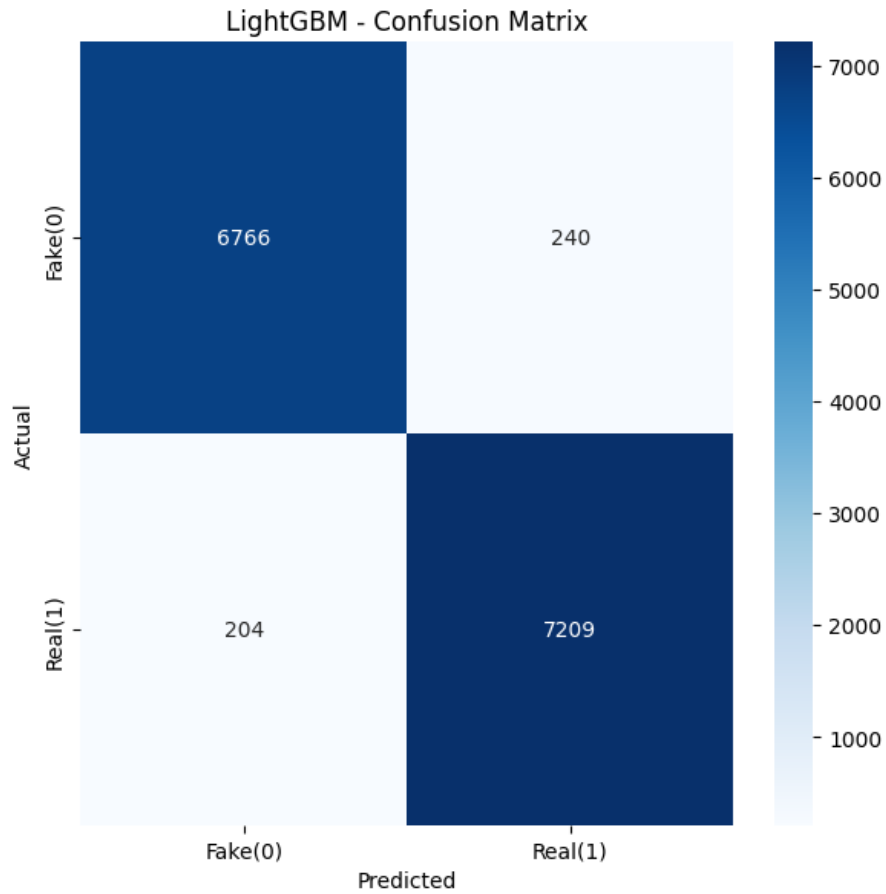


Figure 6.2: LightGBM Confusion Matrix (Test Set)

## 6.1 Hyperparameter Optimization

To maximize the predictive accuracy of the proposed solution and ensure robust generalization, an extensive **hyperparameter tuning** phase was performed on the LightGBM model. We employed a **Grid Search** strategy integrated with stratified **3-fold cross-validation** on the training set to systematically explore the parameter space. The choice to implement a 3-fold Stratified Cross-Validation was driven by the need to balance stochastic stability with computational efficiency.

The optimization process focused on the following key hyperparameters:

- *learning\_rate*: to control the step size at each iteration;
- *num\_leaves* and *max\_depth*: to manage the complexity and depth of the individual trees;
- *n\_estimators*: to determine the total number of boosting rounds;
- *colsample\_bytree* and *subsample*: to implement feature and data sub-sampling, thereby enhancing the model’s resistance to overfitting.

The objective function was set to maximize the **F1-score**, ensuring a balanced trade-off between precision and recall while mitigating the risk of over-specialization on the training data. The grid search space was strategically designed to include the model’s default parameter settings. This approach ensures that the optimization process can directly compare the baseline configuration against alternative hyperparameter sets, guaranteeing that the final selected model meets or exceeds the performance of the out-of-the-box version. The final performance metrics were obtained by retraining the model on the entire training dataset using the optimal hyperparameters. The evaluation was then conducted on the independent **test set** to ensure an unbiased assessment of the model’s generalization capabilities. The results are reported in Table 6.1.

| Metric    | Value  |
|-----------|--------|
| Accuracy  | 0.9797 |
| Precision | 0.9799 |
| Recall    | 0.9795 |
| F1-Score  | 0.9797 |
| AUC-ROC   | 0.9976 |

Table 6.2: Performance Benchmark on the Test Set after Hyperparameter Tuning.

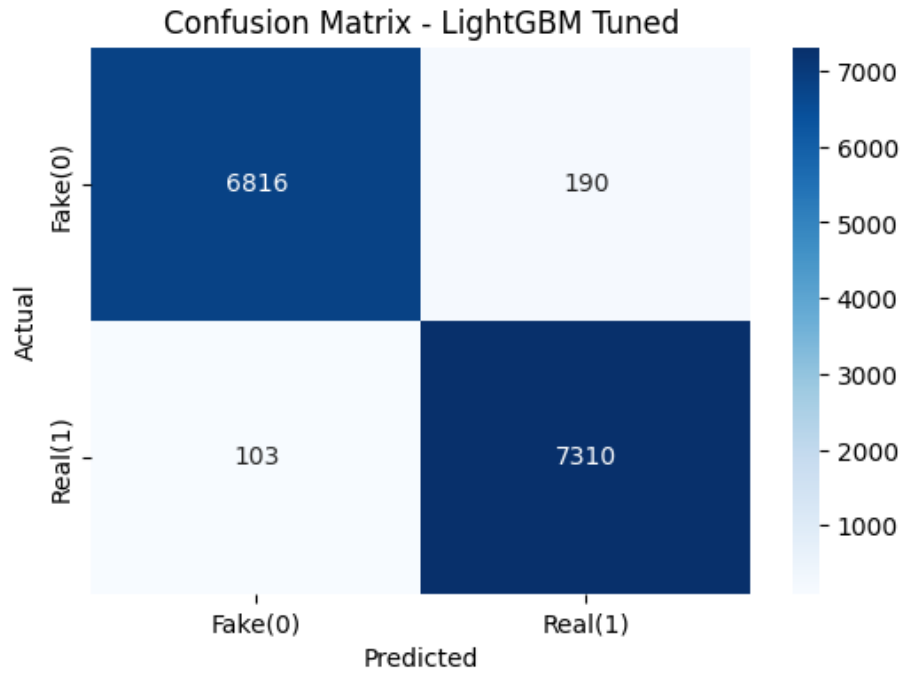


Figure 6.3: LightGBM (after Hyperparameter Tuning) Confusion Matrix (Test Set)

To ensure that the hyperparameter optimization led to a robust and logically sound model, we performed an interpretability analysis using **SHAP**. This post-tuning analysis allows us to verify if the optimized LightGBM model prioritizes features that are semantically relevant to the task of fake news detection.



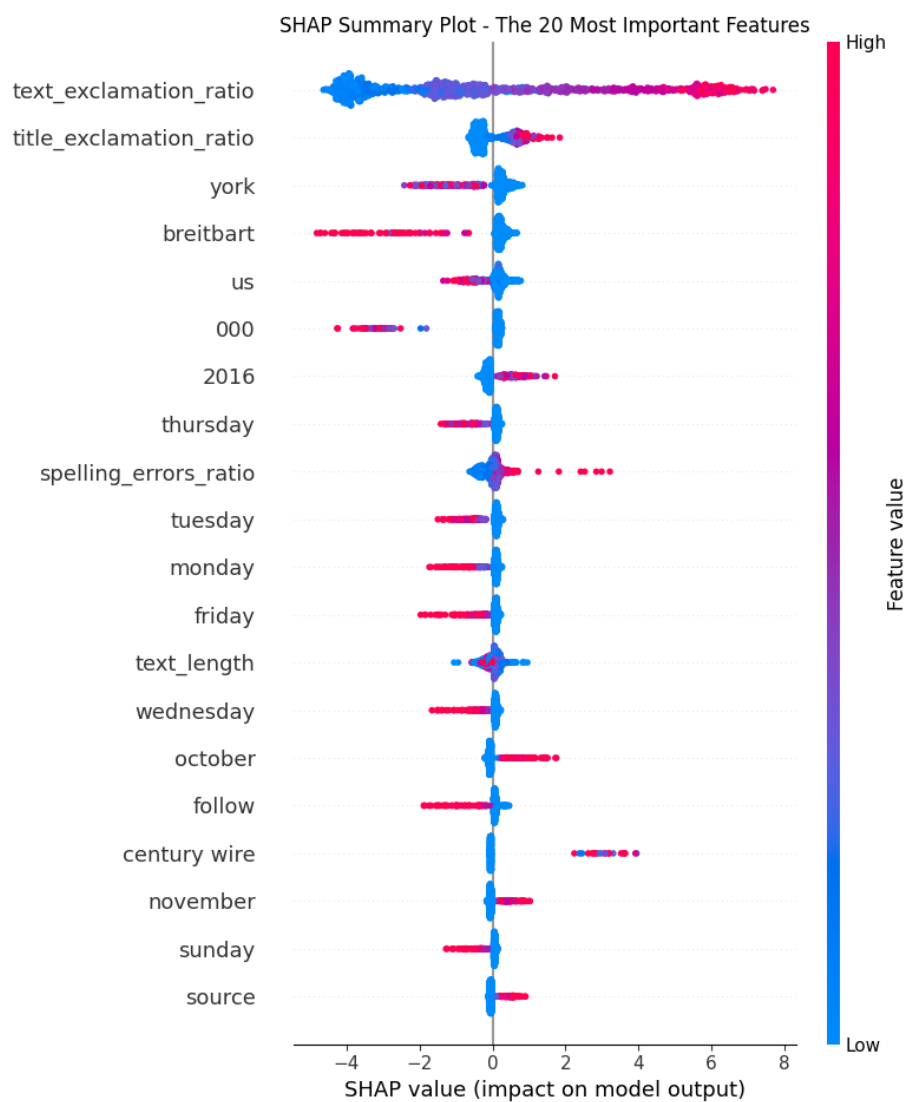


Figure 6.4: SHAP Summary Plot for the LightGBM model after Hyperparameter Tuning, showing feature importance and their impact on model output.

## Chapter 7

# Conclusion and Future Works

### 7.1 Conclusion

The primary objective of this study was to evaluate and compare various machine learning algorithms on a balanced dataset of fake and real news to identify the most reliable model for real-world deployment. Among the candidates evaluated during the benchmarking phase, **LightGBM** emerged as the superior algorithm.

To augment predictive performance and ensure robust generalization, an extensive **hyperparameter optimization** phase was conducted. We employed a **Grid Search** strategy integrated with **Stratified 3-fold cross-validation** on the training data to systematically refine the model's configuration, specifically targeting the maximization of the **F1-score**. This process involved the fine-tuning of critical parameters, including *learning\_rate*, *num\_leaves*, *max\_depth*, and *n\_estimators*, while *colsample\_bytree* and *subsample* were optimized to mitigate the risk of overfitting.

Following this optimization, the fine-tuned LightGBM model was retrained on the complete training set and subsequently evaluated on the **independent test set**. The model achieved slightly better results compared to LightGBM without hyperparameter tuning.

The model's efficacy is largely attributed to its success in processing the **hybrid feature set** designed for this study. By seamlessly integrating semantic representations (TF-IDF) with custom stylometric indicators—such as uppercase ratios, punctuation frequency, and spelling error rates—the tree-based ensemble architecture of LightGBM provided a sophisticated engine for feature fusion. This robust framework serves as the foundation for our final classification tool, achieving the ultimate goal of developing a specialized application for the autonomous and reliable classification of news articles based on their textual and stylistic fingerprints.

Furthermore, the implementation of Explainable AI (XAI) through **SHAP** provided essential insights into the model’s decision-making logic. While the high performance metrics confirm the tool’s effectiveness, the explainability analysis revealed that the model significantly relies on **dataset-specific biases**, such as over-weighting particular named entities (e.g., *breitbart*, *york*) or temporal markers (e.g., *2016*, days of the week).

This finding is crucial as it suggests that while the current application is highly accurate within this specific context, its transition to a universal domain requires further **refinement of the underlying training data**. The objective for future iterations is to move beyond topic-specific patterns and contextual coincidences, focusing instead on a more generalized understanding of **deceptive linguistics** and stylometric features that remain consistent across different news cycles and domains.

## 7.2 Future Works

Despite the promising results obtained, several avenues for future research remain to enhance the application:

- **Transition to Transformer Models:** Future iterations will explore context-aware embeddings like BERT or RoBERTa to capture deeper semantic nuances and reduce the model’s sensitivity to specific keywords.
- **Bias Mitigation:** To improve generalization, the training pipeline should be expanded with more diverse, cross-domain datasets (e.g., healthcare, science, and technology) to ensure the model does not over-fit to political entities.
- **Advanced Web Scraping:** Refining the real-time extraction logic is a priority. Implementing more robust HTML parsing will help isolate the core article body from “web noise” (ads, banners, navigation links), which currently impacts stylometric accuracy.
- **Multilingual Support:** Expanding the classifier to support multiple languages by integrating language-agnostic lemmatization and stylometric extractors.

# Appendix A

## User Manual

**FakeNews Detector** is a desktop application designed to verify the authenticity of news articles. By using the classifier that performed best in the testing phase, the software classifies content as either *Real News* or *Fake news*.

### A.1 Operational Logic

The software follows a strict **Priority Logic** to handle inputs.

### A.2 Online Detection (URL Mode)

If a URL is entered into the designated field, the software attempts to scrape the content. This mode overrides any text manually entered into the offline fields. The system extracts the document  $D$  where:

$$D = \{Title, Body\}$$

This content is then injected back into the UI for user verification before processing.

### A.3 Offline Detection (Manual Mode)

In the absence of a URL, the software reads the **Title** and **Body** fields. If both fields are empty, the system triggers a validation warning.

### A.4 Installation

Ensure Python 3.x is installed. Install dependencies via terminal:

```
pip install requests beautifulsoup4 joblib numpy scipy  
lime nltk pyspellchecker scikit-learn
```

## A.5 Running the Application

To operate the *Fake News Detector*, follow these structured steps:

1. **Initialization:** Launch the application by executing the main script from your terminal or command prompt:

```
python FakeNewsDetectorApp.py
```

2. **Data Input:** Choose one of the two available detection modes:

**Online Mode:** Paste the full article link into the **URL** input field. The system will automatically scrape and populate the **Title** and **Article Body** sections.

**Manual Mode:** Leave the URL field empty and manually type or paste the text into the **Title** and **Article Body** text areas.

3. **Execution:** Click the **Run Detection** button to start the processing.
4. **Interpretation of Results:** The software will display the final verdict at the bottom of the window.
5. **Local Explanation (LIME):** Once the classification is complete, the **VIEW LIME EXPLANATION** button will be enabled. By clicking this button, the system triggers the LIME explainer, which performs a local analysis of the specific article. An interactive HTML report will automatically open in the default web browser, highlighting the specific words and linguistic patterns that most heavily influenced the model's decision.

## A.6 Application Mock-ups

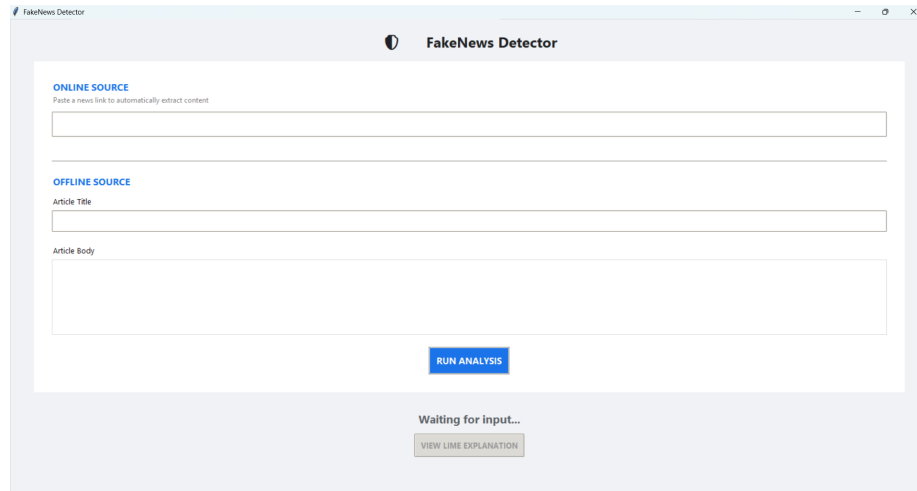


Figure A.1: Application View

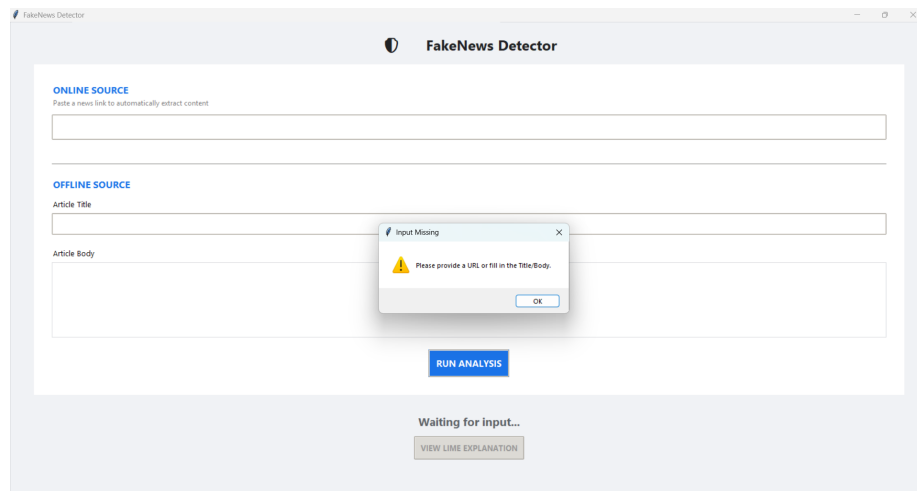


Figure A.2: Input Error

FakeNews Detector

FakeNews Detector

ONLINE SOURCE

Paste a news link to automatically extract content.

https://apnews.com/article/nasa-voyager-1-interstellar-space-1b4a1f9fdaf23d60b539301b3f035b

OFFLINE SOURCE

Article Title

NASA's Voyager 1, the most distant spacecraft from Earth, is doing science again after problem

Article Body

Copyright 2026 The Associated Press. All Rights Reserved.  
Copyright 2026 The Associated Press. All Rights Reserved.

FILE - This illustration provided by NASA depicts Voyager 1. The most distant spacecraft from Earth stopped sending back understandable data in November 2023. The Jet Propulsion Laboratory in Southern California announced this week that Voyager 1's four scientific instruments are back in business after a technical snafu in November. (NASA via AP, file)

RUN ANALYSIS

Scraping content...  

VIEW LIME EXPLANATION

Figure A.3: Web Scraping

 FakeNews Detector

 FakeNews Detector

ONLINE SOURCE

Paste a news link to automatically extract content

OFFLINE SOURCE

Article Title

Article Body

Foreign ministry says remarks of conservative podcast host Michael Savage that were shared by US president were 'obviously uninformed, inappropriate and in poor taste'

India on Thursday criticised as inappropriate a post by Donald Trump in which he shared comments that called the South Asian country a "hellhole".

The inflammatory post on Truth Social comes ahead of a planned visit next month to India by the US secretary of state, Marco Rubio, who is seeking to ease recent tensions between the normally friendly powers.

RUN ANALYSIS

Analyzing...

VIEW LIVE EXPLANATION

Figure A.4: Article Analysis

FakeNews Detector

FakeNews Detector

ONLINE SOURCE

Paste a news link to automatically extract content

<https://apnews.com/article/nasa-voyager-1-interstellar-space-1b4a1f9f9adfa23d60b539301b3f035b>

OFFLINE SOURCE

Article Title

NASA's Voyager 1, the most distant spacecraft from Earth, is doing science again after problem

Article Body

Copyright 2026 The Associated Press. All Rights Reserved.

Copyright 2026 The Associated Press. All Rights Reserved.

FILE - This illustration provided by NASA depicts Voyager 1. The most distant spacecraft from Earth stopped sending back understandable data in November 2023. The Jet Propulsion Laboratory in Southern California announced this week that Voyager 1's four scientific instruments are back in business after a technical snafu in November. (NASA via AP, file)

RUN ANALYSIS

REAL NEWS

VIEW LIME EXPLANATION

FakeNews Detector

FakeNews Detector

ONLINE SOURCE

Paste a news link to automatically extract content

https://www.politifact.com/factchecks/2025/jan/24/tweets/claim-republicans-support-impeaching-president-don/

OFFLINE SOURCE

Article Title

Claim Republicans support impeaching President Donald Trump came from fake news account

Article Body

Our only agenda is to publish the truth so you can be an informed participant in democracy.We need your help.

President Donald Trump holds up an executive order commuting sentences for people convicted of Jan. 6 offenses in the Oval Office of the White House, Jan. 20, 2025, in Washington. (AP)

An X account with a history of sharing false and misleading claims based this claim on unattributed comments it shared in a thread.

RUN ANALYSIS

FAKE NEWS

VUE LIME EXPLANATION



